

Module 1 Hackathon — promptlab

Build the test runner for prompts.

The Situation

Every team that ships prompts hits the same wall. Someone tweaks a prompt on Friday, it looks better on the three examples they tried, and on Monday a customer finds the case it quietly broke. Nobody can say whether v2 is better than v1, because nobody has a way to measure it.

Code has test runners. Prompts mostly have vibes.

Your job is to fix that. Build **promptlab**: a command-line harness that runs a suite of test cases against a prompt, evaluates assertions on what comes back, and reports exactly what changed between two versions of that prompt — including what got worse.

Then you will point it at yourself. One of your deliverables is a prompt you improved, with your own tool as the evidence.

The model you test against

You are given `stubmodel.py`: a local, offline stand-in for an LLM. It is fully documented and you may read it. It is not a puzzle.

```
python stubmodel.py --prompt <file> --input <text|@file>
                    [--temperature 0.0] [--seed N]
                    [--max-tokens 256] [--call-index N]
```

It prints one JSON object to stdout:

```
{"output": "...", "tokens_in": 92, "tokens_out": 24,
 "finish": "stop", "latency_ms": 340}
```

Exit codes: 0 ok · 2 bad arguments · 3 prompt or input file unreadable.

Three things about it will shape your entire design:

1. **At `--temperature 0.0` it is byte-for-byte deterministic.** Same prompt, same input, same output, always.
2. **Above 0.0 it is not.** Output drifts: JSON keys reorder, code fences appear and disappear, preambles slip in, whitespace and casing wobble. With `--seed` it is reproducible; without, it is genuinely random per call.
3. **It responds to prompt quality.** A vague prompt and a well-engineered one produce measurably different results on the same inputs. That is the point. Your harness has to be sensitive enough to see the difference.

Token rule everywhere in this project, including inside your own reports:

```
tokens = math.ceil(len(text) / 4)
```

What You Must Build

The CLI contract — implement exactly

Hidden suites run against this at judging. Deviate and nothing runs.

```
promptlab run      --suite <file> [--runs N] [--out report.json] [--report]
promptlab compare --baseline <report.json> --candidate <report.json> [--out diff.json]
promptlab doctor
```

Exit codes: 0 all cases passed · 1 bad usage or malformed suite · 2 one or more cases failed · 3 the model could not be invoked · 4 a suite or report file was unreadable.

Exit code 2 is a *result*, not an error. Something in CI will depend on that distinction.

The suite format — fixed, you consume it

```
{
  "name": "classify-smoke",
  "prompt_file": "prompts/classify_v1.txt",
  "model": { "temperature": 0.0, "max_tokens": 256 },
  "runs": 1,
  "cases": [
    {
      "id": "c001",
      "input": "I was charged twice for invoice 7",
      "assert": [
        { "type": "json_valid" },
        { "type": "json_field_equals", "field": "category", "value": "billing" },
        { "type": "not_contains", "value": "Sure" },
        { "type": "max_tokens", "value": 40 },
        { "type": "finish_is", "value": "stop" }
      ]
    }
  ]
}
```

`input` is either a string or `{ "file": "path" }`. Paths are relative to the suite file, not the working directory. `--runs` on the command line overrides `runs` in the file.

The eight assertion types — all mandatory

Type	Fields	Passes when
<code>contains</code>	<code>value</code> , optional <code>ignore_case</code>	The output contains the value
<code>not_contains</code>	<code>value</code> , optional <code>ignore_case</code>	It does not
<code>equals</code>	<code>value</code> , optional <code>normalize</code>	Output equals value. <code>normalize</code> strips and collapses whitespace
<code>matches</code>	<code>pattern</code>	The regex finds a match
<code>json_valid</code>	—	The output parses as JSON
<code>json_field_equals</code>	<code>field</code> , <code>value</code>	A dotted path resolves to that value
<code>max_tokens</code>	<code>value</code>	<code>tokens_out</code> is at or below the value
<code>finish_is</code>	<code>value</code>	<code>finish</code> equals <code>stop</code> , <code>length</code> , or <code>refusal</code>

`json_valid` is where your first real decision lives. The model sometimes wraps JSON in a fenced code block. Is fenced JSON valid JSON? There is a defensible answer either way. Pick one, put it in your spec, defend it — and whatever you pick, `json_field_equals` must behave consistently with it.

The report schema — fixed, judges diff it

```
{
  "suite": "classify-smoke",
  "prompt_file": "prompts/classify_v1.txt",
  "prompt_hash": "a19f40cc21b8",
  "runs": 3,
  "model": { "temperature": 0.4, "max_tokens": 256 },
```

```

"totals": {
  "cases": 20, "passed": 16, "failed": 3, "flaky": 1,
  "tokens_in": 4120, "tokens_out": 980, "wall_ms": 8640
},
"cases": [
  {
    "id": "c001",
    "status": "pass",
    "pass_rate": 1.0,
    "tokens_out_avg": 21,
    "assertions": [ { "type": "json_valid", "passed": 3, "failed": 0 } ],
    "failures": []
  }
]
}

```

`prompt_hash` is the first 12 hex characters of the SHA-256 of the prompt file's bytes. Reports are keyed to prompts by this hash, and `compare` uses it to notice when you have compared a report against itself.

Requirements

A — The runner

[MUST 1] Python **standard library only**. Python 3.10+. No third-party packages.

[MUST 2] Load and validate a suite file. A malformed suite fails with a readable message naming what is wrong and where, and exit code 1. Never a traceback.

[MUST 3] Invoke `stubmodel.py` as a **subprocess** for each case. Do not import it, reimplement it, or predict its output. Judges swap in a different model binary with the same CLI contract, and yours must keep working.

[MUST 4] Implement all eight assertion types with the exact semantics above.

[MUST 5] `--runs N` executes each case N times. This is the requirement that matters most here — see section B.

[MUST 6] Emit a report matching the schema exactly. `--out` writes it; without `--out`, the report goes to stdout and human-readable output goes to stderr. Something will pipe this.

[MUST 7] `--report` prints a human summary: pass, fail and flaky counts, total tokens, wall time, and the worst offenders. A person should read it in five seconds.

[MUST 8] Determinism. At temperature 0, running the same suite twice produces byte-identical reports apart from timing. Isolate timing fields so a diff of two runs is clean.

[MUST 9] No crash reaches the user. Missing prompt file, missing input file, model binary absent, invalid regex in a `matches` assertion, unreadable JSON — each produces a one-line message and the right exit code.

B — Non-determinism, which is the real problem

[MUST 10] Above temperature zero the same case does not always give the same answer. Your harness must distinguish three outcomes, not two:

- **pass** — passed every run
- **fail** — failed every run
- **flaky** — passed some, failed others

A case that passes 7 times out of 10 is not a passing case, and reporting it as one is the most common way a prompt harness lies to its owner.

[MUST 11] Report `pass_rate` per case. Your policy for turning a pass rate into a status is **your decision** — threshold, strict unanimity, something else. State it in your spec and defend it.

[MUST 12] Per-assertion counts: for each assertion, how many runs it passed and failed. A case can be flaky because of one assertion out of five, and the report must say which one.

[MUST 13] `failures` carries enough to debug without rerunning: what was asserted, what the output actually was, truncated sensibly. Judges will read this field.

C — Comparison

[MUST 14] `compare` takes two reports and classifies every case: **regressed**, **improved**, **unchanged**, **new**, **removed**.

[MUST 15] A pass rate moving from 1.0 to 0.9 is a regression even if the case is still labelled a pass under your policy. Silent degradation is exactly what this tool exists to catch.

[MUST 16] `compare` reports cost delta: tokens in, tokens out, percentage change. A prompt that gains one point of accuracy for triple the tokens is a decision someone needs to make consciously.

[MUST 17] `compare` warns when the two reports share a `prompt_hash`, come from different suites, or used different model settings. Comparing incomparable things is the most common misuse of a tool like this.

[MUST 18] `doctor` checks the environment and prints a diagnostic: Python version, model binary reachable and responding, suites discoverable, assertion types registered. Judges run this first.

D — Prove it on yourself

[MUST 19] You are given `prompts/classify_v1.txt`. It is deliberately weak. Using your own harness as the measuring instrument, produce `prompts/classify_v2.txt` that measurably outperforms it on the provided ticket corpus.

[MUST 20] `IMPROVEMENT.md` documents the work: baseline report, candidate report, the `compare` output, and — the graded part — **what you changed, what you predicted it would do, and whether you were right**. At least four iterations. Include the change that did *not* help, because there will be one.

This is prompt engineering measured instead of asserted. It is worth more than it looks.

E — Your own tests

[MUST 21] A test suite for the harness itself using `unittest`. Not for the prompts — for your code: assertion evaluation, suite parsing, flaky classification, compare logic.

[MUST 22] `python -m unittest` passes from a fresh clone.

F — Context engineering

[MUST 23] `CLAUDE.md` — the context file you used to drive Claude Code. Graded.

[MUST 24] `PROMPTS.md` — your five most important prompts: what you asked, what came back, what you changed, why.

[MUST 25] `USAGE.md` — documentation written for an **LLM agent** that will run your harness without a human watching. What each command does, when to use it, when not to, what each exit code means and what the agent should do about it. Graded as context engineering, not as documentation.

G — Spec-driven development, enforced

[MUST 26] Your first commit contains `SPEC.md` and no implementation code. Verified with `git log --reverse --stat`. Code in the first commit scores zero on spec discipline.

[MUST 27] `SPEC.md` states, before you build: your assertion evaluation model, your flaky policy and threshold, your fenced-JSON decision, your regression definition, your failure taxonomy, and your definition of done.

[MUST 28] When the design changes, update `SPEC.md` before the code. The history is the evidence.

Stretch

[STRETCH 29] Parallel case execution with deterministic report ordering. [STRETCH 30] A result cache keyed on prompt hash, input and model settings, with a defensible invalidation rule. [STRETCH 31] `--format html` — a standalone report file, stdlib only. [STRETCH 32] Statistical honesty: confidence intervals on pass rates, and a warning when `--runs` is too low to support the conclusion being drawn.

Decisions we are deliberately not making for you

Pick one, record what you rejected, defend it:

1. **Flaky policy.** What pass rate counts as a pass? Is 0.95 a pass at 20 runs and at 2 runs?
2. **Fenced JSON.** Valid or not? Consistency between `json_valid` and `json_field_equals` is what is graded.
3. **Regression threshold.** Is a 0.05 drop noise or a regression? Does the answer depend on `--runs`?
4. **Assertion semantics.** Does the first failing assertion stop the case, or do all assertions always evaluate?
5. **Cost accounting.** Averages per run, or totals across runs? They tell different stories.

Opposite answers can both earn full marks. No answer earns nothing.

Constraints

- Python standard library only. No `pytest`, no `yaml`, no `rich`.
- No network access. Everything is local.
- **Claude Code is required.** Your context files and prompts are graded artifacts.
- Do not import, copy or reimplement `stubmodel.py`. Subprocess only.
- Fresh clone to `doctor` plus a passing smoke suite in under five minutes, following your README.

What You Submit

Deliverable	Notes
Git repo, full history	First commit: <code>SPEC.md</code> only
<code>SPEC.md</code>	Written first, updated as you learn
<code>promptlab</code>	Working CLI, stdlib only
<code>prompts/classify_v2.txt</code>	Your improved prompt
<code>IMPROVEMENT.md</code>	Baseline, candidate, compare output, four-plus iterations
<code>tests/</code>	<code>unittest</code> , passing
<code>USAGE.md</code>	Agent-facing documentation
<code>CLAUDE.md</code> , <code>PROMPTS.md</code>	How you drove the assistant
<code>JOURNAL.md</code>	One page, five questions

Deliverable	Notes
3-minute live demo	At judging

JOURNAL.md

1. Three decisions we made, and what we rejected in each case.
2. The hardest bug we hit, and how we found the root cause.
3. **Something Claude Code got confidently wrong, and how we caught it.**
4. What we would do differently with four more hours.
5. Who did what — per person.

How You'll Be Judged

Criterion	Weight
Functionality — hidden suites, assertions, flaky handling, compare	30
Understanding — the live viva	20
Prompt improvement — IMPROVEMENT.md and the measured gain	15
Curveball response	15
Spec discipline, tests and context artifacts	20

Understanding gates everything. If no member of your team can explain a component, that component scores zero even if it works.

Hidden suites — the categories

At judging your harness runs suites you have never seen. You get the categories, not the cases:

- An empty suite, and a suite with one case and zero assertions
- A malformed suite — missing fields, wrong types, unknown assertion type
- A suite pointing at a prompt file that does not exist
- A high-temperature suite with `--runs 10`, checked for correct flaky classification
- A suite where every case fails, checked for exit code
- Invalid regex in `matches`, and assertions on deeply nested JSON paths
- A case truncated mid-JSON by `--max-tokens`
- A case that triggers a model refusal
- Repeat runs at temperature 0, diffed byte for byte
- Two reports from genuinely different suites, passed to `compare`
- **A different model binary** with the same CLI contract and different behavior

Read that last one twice. If your harness assumes anything about what the model says rather than what the contract *guarantees*, it fails there.

Rules on AI Use

Claude Code is **required**. Use it as hard as you can.

And: **you own every line you submit**. Judges pick the team member and judges pick the file. *"The AI wrote it"* is not an answer, and a team that cannot explain its own harness loses the entire Understanding score regardless of whether it works.

There is a particular trap in this project. Ask an assistant to build a prompt test runner and it produces something clean, plausible, and quietly wrong in one specific way: it treats every case as pass or fail, because that is what test runners do. Non-determinism is the whole problem here, and the pattern-matched answer does not account for it.

Watch for the moment you notice that. Then write it down for question 3.

Timeline — 24 hours

Time	Event
0:00	Kickoff, brief released, starter materials handed out
0:00 – 2:00	Spec phase. <code>SPEC.md</code> committed. No implementation code.
2:00	Checkpoint 1 — mentors review the spec, especially your flaky policy
6:00	Checkpoint 2 — <code>run</code> executes one suite end to end, however thin
12:00	Checkpoint 3 — <code>compare</code> works; <code>IMPROVEMENT.md</code> has started
14:00	Curveball released — same for every team, live and in writing
15:00	Checkpoint 4 — you have a plan for the curveball
21:00	Checkpoint 5 — fresh clone: <code>doctor</code> , smoke suite, <code>python -m unittest</code>
23:00	Submission freeze
23:00 →	Judging: <code>doctor</code> , demo, hidden suites, viva

There is a required change to the specification at 14:00. Its content is not announced in advance. Architecture that absorbs it cleanly is architecture that was understood.

Starter Materials

- `stubmodel.py` — the model, fully documented
- `prompts/classify_v1.txt` — the weak prompt you must improve
- `data/tickets.json` — 63 tickets with ground-truth labels
- `suites/smoke.json` — one small suite in the correct format
- `SPEC_TEMPLATE.md`, `README_TEMPLATE.md`

Scaling

Duration	Scope
10 hours	MUSTs 1–18 and 26–27. Drop the improvement track and your own tests.
24 hours	This brief as written.
48 hours	Add all stretch items, plus a second model binary with different behavior that one harness must support.

One Last Thing

You have spent this module learning to write prompts and shape context, and judging your prompts the way everybody does — looking at a few outputs and deciding they seem better.

Today you build the instrument that tells you whether they actually are. Then you point it at your own work.

Some of you will find out that a prompt you were proud of was worse.